Deep Learning Lab

Complete Algorithm & Code Reference

Rohith Dharshan M | 23N241

PART 1 — LAB EXPERIMENTS

Experiment 1 — Study of TensorFlow & Keras

Algorithm

Step 1: Understand TensorFlow architecture stack (Keras API → C++ Runtime → Hardware).
Step 2: Study Tensors — Rank 0 (Scalar), Rank 1 (Vector), Rank 2 (Matrix), Rank 3+ (n-D array).
Step 3: Understand Tensor Graphs — Nodes are operations, Edges carry data.
Step 4: Study Tensor Operations — element-wise, matrix multiplication, broadcasting.
Step 5: Create and manipulate tensors using TensorFlow/Keras API.
Step 6: Observe shape, dtype, and rank of tensors.

Code

```
import tensorflow as tf
import numpy as np

a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
b = tf.constant([[5, 6], [7, 8]], dtype=tf.float32)

print("Tensor a:", a)
print("Shape:", a.shape)
print("Rank:", tf.rank(a).numpy())
print("Dtype:", a.dtype)

add = tf.add(a, b)
mul = tf.matmul(a, b)
print("Element-wise add:", add.numpy())
print("Matrix multiply:", mul.numpy())

scalar = tf.constant(3.14)
vector = tf.constant([1.0, 2.0, 3.0])
matrix = tf.constant([[1, 2], [3, 4]])
tensor3d = tf.zeros([2, 3, 4])
print("Scalar rank:", tf.rank(scalar).numpy())
print("Vector rank:", tf.rank(vector).numpy())
print("Matrix rank:", tf.rank(matrix).numpy())
print("3D Tensor shape:", tensor3d.shape)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(4, activation='relu', input_shape=(2,)),
    Dense(1, activation='sigmoid')
])
model.summary()
```

Experiment 2 — Single Layer Perceptron for Logic Gates

Algorithm

Step 1: Initialize weights w1, w2 and bias to zero.
Step 2: Select dataset for the chosen logic gate (AND / OR / NAND).
Step 3: For each input compute: net_input = w1*x1 + w2*x2 + bias.
Step 4: Apply step activation: output = 1 if net_input >= 0 else 0.
Step 5: Compute error = target - output.
Step 6: Update: w1 += lr * error * x1; w2 += lr * error * x2; bias += lr * error.
Step 7: Repeat for all epochs until weights converge.
Step 8: Test using all input combinations and display results.

Code

```
def step_activation(value):
    if value >= 0:
        return 1
    else:
        return 0

class Perceptron:
    def __init__(self, learning_rate, epochs):
        self.lr = learning_rate
        self.epochs = epochs
        self.w1 = 0
        self.w2 = 0
        self.bias = 0

    def train(self, inputs, targets):
        for _ in range(self.epochs):
            for i in range(len(inputs)):
                x1, x2 = inputs[i]
                target = targets[i]
                net_input = (self.w1 * x1) + (self.w2 * x2) + self.bias
                output = step_activation(net_input)
                error = target - output
                self.w1 += self.lr * error * x1
                self.w2 += self.lr * error * x2
                self.bias += self.lr * error

    def predict(self, inputs):
        results = []
        for x1, x2 in inputs:
            net_input = (self.w1 * x1) + (self.w2 * x2) + self.bias
            results.append(step_activation(net_input))
        return results

inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
learning_rate = float(input("Enter learning rate: "))
epochs = int(input("Enter epochs: "))
print("\nChoose Logic Gate: 1.AND  2.OR  3.NAND")
choice = int(input("Enter choice (1/2/3): "))

if choice == 1:
    targets = [0, 0, 0, 1]; gate_name = "AND"
elif choice == 2:
    targets = [0, 1, 1, 1]; gate_name = "OR"
elif choice == 3:
    targets = [1, 1, 1, 0]; gate_name = "NAND"

perceptron = Perceptron(learning_rate, epochs)
perceptron.train(inputs, targets)
outputs = perceptron.predict(inputs)
```

```
print(f"\n{gate_name} Gate Output:")
for i in range(len(inputs)):
    print(f"{inputs[i]} -> {outputs[i]}")
```

Experiment 3 — Multi Layer Perceptron for XOR (From Scratch)

Algorithm

Step 1: Initialize weights (w1[2][2], w2[2]) and biases (b1[2], b2) randomly.
Step 2: Repeat for all epochs:
  For each XOR sample (x1, x2):
  Forward Pass: compute hidden h1, h2 using sigmoid; compute output using sigmoid.
  Backward Pass: compute output gradient d_out; propagate to hidden d_h1, d_h2.
  Update all weights and biases using gradient descent.
Step 3: After training, predict and display results for all XOR inputs.

Code

```
import math
import random

def sig(x):
    return 1 / (1 + math.exp(-max(min(x, 45), -45)))

def dsig(y):
    return y * (1 - y)

X = [(0, 0), (0, 1), (1, 0), (1, 1)]
Y = [0, 1, 1, 0]
random.seed(42)

w1 = [[random.random(), random.random()], [random.random(), random.random()]]
b1 = [random.random(), random.random()]
w2 = [random.random(), random.random()]
b2 = random.random()
lr = 1.0

for epoch in range(5000):
    for i in range(4):
        x1, x2 = X[i]
        y = Y[i]
        h1 = sig(x1*w1[0][0] + x2*w1[1][0] + b1[0])
        h2 = sig(x1*w1[0][1] + x2*w1[1][1] + b1[1])
        out = sig(h1*w2[0] + h2*w2[1] + b2)
        err = y - out
        d_out = err * dsig(out)
        d_w2_0 = d_out * h1
        d_w2_1 = d_out * h2
        d_h1 = d_out * w2[0] * dsig(h1)
        d_h2 = d_out * w2[1] * dsig(h2)
        w2[0] += lr * d_w2_0
        w2[1] += lr * d_w2_1
        b2 += lr * d_out
        w1[0][0] += lr * d_h1 * x1
        w1[1][0] += lr * d_h1 * x2
        b1[0] += lr * d_h1
        w1[0][1] += lr * d_h2 * x1
        w1[1][1] += lr * d_h2 * x2
        b1[1] += lr * d_h2

print("Input      Predicted")
print("-" * 25)
for x in X:
    x1, x2 = x
    h1 = sig(x1*w1[0][0] + x2*w1[1][0] + b1[0])
    h2 = sig(x1*w1[0][1] + x2*w1[1][1] + b1[1])
    out = sig(h1*w2[0] + h2*w2[1] + b2)
    print(f"({x1}, {x2}) -> {out:.3f} ({round(out)})")
```

Experiment 4 — CNN with Max Pooling for MNIST

Algorithm

Step 1: Load MNIST dataset (60,000 train + 10,000 test images).
Step 2: Normalize pixel values to [0, 1] and reshape to (28, 28, 1).
Step 3: One-hot encode labels (10 classes).
Step 4: Build CNN: Conv2D(32) → MaxPool → Conv2D(64) → MaxPool → Flatten → Dense(128) → Dense(10, softmax).
Step 5: Compile with Adam optimizer and categorical crossentropy loss.
Step 6: Train for 5 epochs with batch size 32.
Step 7: Evaluate on test set and print accuracy.

Code

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import to_categorical

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train / 255.0
X_test = X_test / 255.0
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=5, batch_size=32)
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)
```

Experiment 6 — SLP, MLP, CNN Using Existing Packages

Algorithm

SLP: Import sklearn Perceptron → fit on AND gate data → predict and print weights.

MLP: Define PyTorch Sequential model (Linear→Sigmoid→Linear→Sigmoid) → train with BCELoss + SGD for 5000 epochs → print predictions.

CNN: Load MNIST with TensorFlow → normalize & reshape → build Conv2D+Pooling+Dense model → compile → train 3 epochs → evaluate.

Code — Program 1: Single Layer Perceptron (SLP)

```
from sklearn.linear_model import Perceptron
import numpy as np

X = np.array([[0,0], [0,1], [1,0], [1,1]])
y = np.array([0, 0, 0, 1])

model = Perceptron(max_iter=100, eta0=0.1, random_state=0)
model.fit(X, y)

print("Predictions:", model.predict(X))
print("Final Weights:", model.coef_)
print("Final Bias:", model.intercept_)
```

Code — Program 2: Multi-Layer Perceptron (MLP)

```
import os
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

import torch
import torch.nn as nn
import torch.optim as optim

X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
y = torch.tensor([[0],[1],[1],[0]], dtype=torch.float32)

model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid()
)

criterion = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

for epoch in range(5000):
    optimizer.zero_grad()
    outputs = model(X)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()

print("Predictions:")
print(model(X).detach().numpy().round())
```

Code — Program 3: Convolutional Neural Network (CNN)

```
import tensorflow as tf
from tensorflow.keras import layers, models

mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)
```

```
model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=3, batch_size=32)
model.evaluate(x_test, y_test)
```

Experiment 7 — Optimization Algorithms: SGD, Adam, RMSProp

Algorithm

Step 1: Generate dataset: X random, y = 3X + 2 + noise.
Step 2: Initialize weights w and bias b randomly.
Step 3 — SGD: Compute predictions, error, gradients; update w -= lr*dw; b -= lr*db.
Step 4 — RMSProp: Maintain running average v_w; update w -= lr*dw / (sqrt(v_w) + eps).
Step 5 — Adam: Maintain first moment m_w and second moment v_w; apply bias correction; update weight.
Step 6: Train all three for 100 epochs and compare final weights.

Code

```
import numpy as np

np.random.seed(0)
X = np.random.randn(100, 1)
y = 3*X + 2 + np.random.randn(100, 1)*0.1

w = np.random.randn(1, 1)
b = np.zeros((1, 1))
lr = 0.01
epochs = 100

w_sgd = w.copy()
b_sgd = b.copy()
for epoch in range(epochs):
    y_pred = X*w_sgd + b_sgd
    error = y_pred - y
    dw = np.mean(error*X)
    db = np.mean(error)
    w_sgd -= lr*dw
    b_sgd -= lr*db

w_rms = w.copy()
b_rms = b.copy()
beta = 0.9
epsilon = 1e-8
v_w = 0; v_b = 0
for epoch in range(epochs):
    y_pred = X*w_rms + b_rms
    error = y_pred - y
    dw = np.mean(error*X)
    db = np.mean(error)
    v_w = beta*v_w + (1-beta)*(dw**2)
    v_b = beta*v_b + (1-beta)*(db**2)
    w_rms -= lr*dw/(np.sqrt(v_w)+epsilon)
    b_rms -= lr*db/(np.sqrt(v_b)+epsilon)

w_adam = w.copy()
b_adam = b.copy()
m_w = 0; v_w = 0; m_b = 0; v_b = 0
beta1 = 0.9; beta2 = 0.999
for t in range(1, epochs+1):
    y_pred = X*w_adam + b_adam
    error = y_pred - y
    dw = np.mean(error*X)
    db = np.mean(error)
    m_w = beta1*m_w + (1-beta1)*dw
    v_w = beta2*v_w + (1-beta2)*(dw**2)
    m_b = beta1*m_b + (1-beta1)*db
    v_b = beta2*v_b + (1-beta2)*(db**2)
    m_w_hat = m_w/(1-beta1**t)
    v_w_hat = v_w/(1-beta2**t)
```

```
    m_b_hat = m_b/(1-beta1**t)
    v_b_hat = v_b/(1-beta2**t)
    w_adam -= lr*m_w_hat/(np.sqrt(v_w_hat)+1e-8)
    b_adam -= lr*m_b_hat/(np.sqrt(v_b_hat)+1e-8)

print("SGD Weights:", w_sgd)
print("RMSProp Weights:", w_rms)
print("Adam Weights:", w_adam)
```

Experiment 8 — Transfer Learning (VGG16, ResNet50, MobileNetV2)

Algorithm

Step 1: Load image dataset using ImageDataGenerator (rescale, 80/20 split).

Step 2: For each base model (VGG16, ResNet50, MobileNetV2): load with imagenet weights, exclude top layer.

Step 3: Freeze all pretrained layers.

Step 4: Add GlobalAveragePooling → Dense(64, relu) → Dense(num_classes, softmax).

Step 5: Compile with Adam + categorical crossentropy.

Step 6: Train for 3 epochs and compare validation accuracy across models.

Code

```
import tensorflow as tf
from tensorflow.keras.applications import VGG16, ResNet50, MobileNetV2
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator

dataset_path = "sample_flowers/"
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_data = datagen.flow_from_directory(
    dataset_path, target_size=(160,160), batch_size=8,
    class_mode='categorical', subset='training')

val_data = datagen.flow_from_directory(
    dataset_path, target_size=(160,160), batch_size=8,
    class_mode='categorical', subset='validation')

num_classes = train_data.num_classes

def build_model(base_model):
    for layer in base_model.layers:
        layer.trainable = False
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(64, activation='relu')(x)
    output = Dense(num_classes, activation='softmax')(x)
    model = Model(inputs=base_model.input, outputs=output)
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

base_vgg = VGG16(weights='imagenet', include_top=False, input_shape=(160,160,3))
model_vgg = build_model(base_vgg)
history_vgg = model_vgg.fit(train_data, validation_data=val_data, epochs=3)

base_resnet = ResNet50(weights='imagenet', include_top=False, input_shape=(160,160,3))
model_resnet = build_model(base_resnet)
history_resnet = model_resnet.fit(train_data, validation_data=val_data, epochs=3)

base_mobile = MobileNetV2(weights='imagenet', include_top=False, input_shape=(160,160,3))
model_mobile = build_model(base_mobile)
history_mobile = model_mobile.fit(train_data, validation_data=val_data, epochs=3)
```

Experiment 9 — Face Recognition Using SSD

Algorithm

Step 1: Import OpenCV and NumPy.
Step 2: Load pretrained SSD model from deploy.prototxt and caffemodel weights.
Step 3: Read input image and get its height and width.
Step 4: Convert image to blob (resize to 300×300, apply mean subtraction).
Step 5: Pass blob to network via setInput(); run net.forward() to get detections.
Step 6: Loop through detections; if confidence > 0.5, draw bounding box and label.
Step 7: Display output image with detected faces.

Code

```
import cv2
import numpy as np

prototxt = "deploy.prototxt"
model = "res10_300x300_ssd_iter_140000.caffemodel"
net = cv2.dnn.readNetFromCaffe(prototxt, model)

image = cv2.imread("face.jpg")
(h, w) = image.shape[:2]

blob = cv2.dnn.blobFromImage(
    cv2.resize(image, (300, 300)), 1.0, (300, 300), (104.0, 177.0, 123.0))

net.setInput(blob)
detections = net.forward()

for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]
    if confidence > 0.5:
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        (startX, startY, endX, endY) = box.astype("int")
        cv2.rectangle(image, (startX, startY), (endX, endY), (0, 255, 0), 2)
        text = "Face: {:.2f}%".format(confidence * 100)
        cv2.putText(image, text, (startX, startY-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

cv2.imshow("Face Detection using SSD", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Experiment 10 — Time Series Prediction: RNN, LSTM, GRU

Algorithm

Step 1: Generate sine wave time series data (200 points).
Step 2: Create sliding window sequences of length 5.
Step 3: Reshape to (samples, timesteps, 1) and split 80/20 train/test.
Step 4: Build RNN model: SimpleRNN(50, tanh) → Dense(1); compile with Adam + MSE.
Step 5: Build LSTM model: LSTM(50, tanh) → Dense(1); compile same.
Step 6: Build GRU model: GRU(50, tanh) → Dense(1); compile same.
Step 7: Train all three for 10 epochs; predict on test set; plot all vs actual.

Code

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, LSTM, GRU

np.random.seed(0)
data = np.sin(np.linspace(0, 50, 200))

def create_dataset(data, time_step=5):
    X, y = [], []
    for i in range(len(data)-time_step):
        X.append(data[i:i+time_step])
        y.append(data[i+time_step])
    return np.array(X), np.array(y)

time_step = 5
X, y = create_dataset(data, time_step)
X = X.reshape(X.shape[0], X.shape[1], 1)

train_size = int(len(X)*0.8)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

model_rnn = Sequential([SimpleRNN(50, activation='tanh', input_shape=(time_step,1)),
Dense(1)])
model_rnn.compile(optimizer='adam', loss='mse')
model_rnn.fit(X_train, y_train, epochs=10, verbose=0)

model_lstm = Sequential([LSTM(50, activation='tanh', input_shape=(time_step,1)), Dense(1)])
model_lstm.compile(optimizer='adam', loss='mse')
model_lstm.fit(X_train, y_train, epochs=10, verbose=0)

model_gru = Sequential([GRU(50, activation='tanh', input_shape=(time_step,1)), Dense(1)])
model_gru.compile(optimizer='adam', loss='mse')
model_gru.fit(X_train, y_train, epochs=10, verbose=0)

pred_rnn = model_rnn.predict(X_test)
pred_lstm = model_lstm.predict(X_test)
pred_gru = model_gru.predict(X_test)

plt.plot(y_test, label="Actual")
plt.plot(pred_rnn, label="RNN")
plt.plot(pred_lstm, label="LSTM")
plt.plot(pred_gru, label="GRU")
plt.legend()
plt.show()
```

Experiment 11 — Transformer Architecture From Scratch

Algorithm

Step 1: Create random input sequence (seq_len=4, d_model=8).
Step 2: Add Positional Encoding using sine/cosine functions.
Step 3: Compute Q, K, V by multiplying X with learned weight matrices Wq, Wk, Wv.
Step 4: Scaled Dot-Product Attention: scores = $Q \cdot K^T / \sqrt{d_k}$; weights = softmax(scores); output = weights · V.
Step 5: Feed-forward: ReLU(X·W1)·W2.
Step 6: Add residual connections and layer normalization.
Step 7: Print transformer output.

Code

```
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum(axis=-1, keepdims=True)

def attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = np.matmul(Q, K.T) / np.sqrt(d_k)
    weights = softmax(scores)
    return np.matmul(weights, V)

def positional_encoding(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i] = np.sin(pos / (10000**(i/d_model)))
            PE[pos, i+1] = np.cos(pos / (10000**(i/d_model)))
    return PE

seq_len = 4
d_model = 8
X = np.random.rand(seq_len, d_model)
X = X + positional_encoding(seq_len, d_model)

Wq = np.random.rand(d_model, d_model)
Wk = np.random.rand(d_model, d_model)
Wv = np.random.rand(d_model, d_model)

Q = np.dot(X, Wq)
K = np.dot(X, Wk)
V = np.dot(X, Wv)

output = attention(Q, K, V)
print("Transformer Output:\n", output)
```

Experiment 12 — Autoencoder for Video Frame Reconstruction

Algorithm

Step 1: Upload video file (or use dummy data if no video).
Step 2: Extract frames using OpenCV; resize to 64×64; normalize to [0,1].
Step 3: Flatten each frame to a 1D vector (input_dim = 64×64×3 = 12288).
Step 4: Build autoencoder: Input → Dense(128, relu) → Dense(input_dim, sigmoid).
Step 5: Compile with Adam + MSE; train with input = output for 5 epochs.
Step 6: Reconstruct frames by passing through autoencoder.
Step 7: Reshape reconstructed vectors back to frames and save as video.

Code

```
from google.colab import files
uploaded = files.upload()
video_path = list(uploaded.keys())[0]

import cv2
import numpy as np
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

cap = cv2.VideoCapture(video_path)
frames = []

if not cap.isOpened():
    print("Error: Cannot open video")
else:
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame = cv2.resize(frame, (64, 64))
        frame = frame / 255.0
        frames.append(frame)
    cap.release()

if len(frames) == 0:
    frames = np.random.rand(100, 64, 64, 3)

frames = np.array(frames)
X = frames.reshape(len(frames), -1)

input_dim = X.shape[1]
encoding_dim = 128

input_layer = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation='relu')(input_layer)
decoded = Dense(input_dim, activation='sigmoid')(encoded)

autoencoder = Model(input_layer, decoded)
autoencoder.compile(optimizer='adam', loss='mse')
autoencoder.fit(X, X, epochs=5, batch_size=16)

reconstructed = autoencoder.predict(X)
reconstructed = reconstructed.reshape(-1, 64, 64, 3)

out = cv2.VideoWriter("output.avi", cv2.VideoWriter_fourcc(*'XVID'), 10, (64, 64))
for frame in reconstructed:
    frame = (frame * 255).astype(np.uint8)
    out.write(frame)
out.release()

files.download("output.avi")
```

Experiment 13 — Recommender System Using RBM

Algorithm

Step 1: Create 100×20 ratings matrix; each user rates 8–15 movies (1–5); rest = 0.
Step 2: Normalize: scale 1–5 → 0.2–1.0; keep 0 as 0.
Step 3: Split 80% train / 20% test.
Step 4: RBM architecture: visible = 20 units (movies), hidden = 10 units (latent features).
Step 5: Training (CD-1): positive phase P(h|v); negative phase reconstruct v′; update W, biases.
Step 6: For recommendation: pass user vector → reconstruct → pick top-N from unrated movies.

Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

np.random.seed(42)
n_users = 100
n_movies = 20
movie_names = [
    "Inception","The Matrix","Interstellar","The Dark Knight","Avengers",
    "Titanic","Forrest Gump","The Godfather","Pulp Fiction","Fight Club",
    "Goodfellas","Schindler's List","The Shawshank Redemption","Parasite",
    "Joker","1917","Dune","Oppenheimer","Barbie","Top Gun: Maverick"]

ratings = np.zeros((n_users, n_movies))
for u in range(n_users):
    rated = np.random.choice(n_movies, size=np.random.randint(8, 16), replace=False)
    ratings[u, rated] = np.random.randint(1, 6, size=len(rated))

class RBM:
    def __init__(self, n_visible, n_hidden, learning_rate=0.05, n_epochs=50, batch_size=10):
        self.nv = n_visible
        self.nh = n_hidden
        self.lr = learning_rate
        self.epochs = n_epochs
        self.batch_size = batch_size
        self.W = np.random.normal(0, 0.01, (n_visible, n_hidden))
        self.bv = np.zeros(n_visible)
        self.bh = np.zeros(n_hidden)
        self.train_errors = []

    def _sigmoid(self, x):
        return 1.0 / (1.0 + np.exp(-np.clip(x, -30, 30)))

    def _sample_hidden(self, v):
        prob = self._sigmoid(v @ self.W + self.bh)
        sample = (np.random.rand(*prob.shape) < prob).astype(float)
        return prob, sample

    def _sample_visible(self, h):
        prob = self._sigmoid(h @ self.W.T + self.bv)
        sample = (np.random.rand(*prob.shape) < prob).astype(float)
        return prob, sample

    def train(self, data):
        n_samples = data.shape[0]
        for epoch in range(1, self.epochs + 1):
            np.random.shuffle(data)
            epoch_error = 0.0
            n_batches = 0
            for start in range(0, n_samples, self.batch_size):
                batch = data[start : start + self.batch_size]
```

```
                ph0, h0 = self._sample_hidden(batch)
                pv1, v1 = self._sample_visible(h0)
                ph1, _ = self._sample_hidden(pv1)
                bs = batch.shape[0]
                self.W += self.lr * (batch.T @ ph0 - pv1.T @ ph1) / bs
                self.bv += self.lr * (batch - pv1).mean(axis=0)
                self.bh += self.lr * (ph0 - ph1).mean(axis=0)
                epoch_error += np.mean((batch - pv1) ** 2)
                n_batches += 1
            self.train_errors.append(epoch_error / n_batches)

    def reconstruct(self, v):
        ph, h = self._sample_hidden(v)
        pv, _ = self._sample_visible(ph)
        return pv

    def recommend(self, user_vector, top_n=5, movie_names=None):
        reconstructed = self.reconstruct(user_vector.reshape(1, -1))[0]
        unrated_mask = (user_vector == 0)
        scores = reconstructed * unrated_mask
        top_idx = np.argsort(scores)[::-1][:top_n]
        if movie_names:
            return [(movie_names[i], round(float(scores[i]), 3)) for i in top_idx if
scores[i] > 0]
        return top_idx, scores[top_idx]

def normalise(matrix):
    norm = matrix.copy().astype(float)
    norm[norm > 0] = norm[norm > 0] / 5.0
    return norm

train_data, test_data = train_test_split(ratings, test_size=0.2, random_state=42)
train_norm = normalise(train_data)
test_norm = normalise(test_data)

rbm = RBM(n_visible=n_movies, n_hidden=10, learning_rate=0.05, n_epochs=50, batch_size=10)
rbm.train(train_norm)

for uid in range(3):
    recs = rbm.recommend(test_norm[uid], top_n=5, movie_names=movie_names)
    print(f"Test User {uid} Recommendations:")
    for rank, (title, score) in enumerate(recs, 1):
        print(f"  {rank}. {title} — score={score:.3f}")
```

PART 2 — QUESTION BANK SOLUTIONS

Q1 — Single Perceptron Regression (House Price Prediction)

Algorithm

Step 1: Define dataset: Area X = [500,800,1000,1200,1500], Price Y = [150,220,300,360,450].
Step 2: Normalize X and Y by their max values.
Step 3: Initialize weight w=0, bias b=0.
Step 4: For 1000 epochs: predict = w*X + b; compute MSE loss; compute gradients dw, db; update w and b.
Step 5: Predict price for 1100 sqft using trained w and b (then denormalize).
Step 6: Plot loss reduction and regression line.

Code

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([500, 800, 1000, 1200, 1500], dtype=float)
Y = np.array([150, 220, 300, 360, 450], dtype=float)
X_norm = X / X.max()
Y_norm = Y / Y.max()

w = 0.0
b = 0.0
lr = 0.01
epochs = 1000
losses = []

for epoch in range(epochs):
    y_pred = w * X_norm + b
    loss = np.mean((y_pred - Y_norm) ** 2)
    losses.append(loss)
    dw = np.mean((y_pred - Y_norm) * X_norm)
    db = np.mean(y_pred - Y_norm)
    w -= lr * dw
    b -= lr * db

x_new = 1100 / X.max()
predicted = (w * x_new + b) * Y.max()
print(f"Predicted price for 1100 sqft: {predicted:.2f} thousand")

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(losses)
plt.title("Loss Reduction")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")

plt.subplot(1, 2, 2)
plt.scatter(X, Y, color='blue', label='Actual')
x_line = np.linspace(400, 1600, 100)
y_line = (w * (x_line / X.max()) + b) * Y.max()
plt.plot(x_line, y_line, color='red', label='Regression Line')
plt.xlabel("Area (sqft)")
plt.ylabel("Price (thousands)")
plt.legend()
plt.title("Regression Line")
plt.tight_layout()
plt.show()
```

Q2 — MLP: Student Pass/Fail Classification

Algorithm

Step 1: Define dataset: Study Hours & Attendance as inputs; Pass/Fail as output.
Step 2: Normalize inputs (divide by max values).
Step 3: Initialize W1 (2→3), b1, W2 (3→1), b2 randomly.
Step 4: For 50 epochs:
  Forward: z1 = X·W1 + b1; a1 = sigmoid(z1); z2 = a1·W2 + b2; a2 = sigmoid(z2).
  Backward: compute gradients from output back to input layer.
  Update all weights and biases.
Step 5: Compute accuracy; predict for (5 hrs, 65% attendance).

Code

```
import numpy as np

X = np.array([[2,50],[4,60],[6,70],[8,80]], dtype=float)
Y = np.array([[0],[0],[1],[1]], dtype=float)
X = X / np.array([10, 100])

np.random.seed(42)
W1 = np.random.randn(2, 3) * 0.1
b1 = np.zeros((1, 3))
W2 = np.random.randn(3, 1) * 0.1
b2 = np.zeros((1, 1))

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_deriv(x):
    return x * (1 - x)

lr = 0.1
for epoch in range(50):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = sigmoid(z2)
    loss = np.mean((Y - a2) ** 2)
    d_a2 = -(Y - a2)
    d_z2 = d_a2 * sigmoid_deriv(a2)
    d_W2 = a1.T @ d_z2
    d_b2 = np.sum(d_z2, axis=0, keepdims=True)
    d_a1 = d_z2 @ W2.T
    d_z1 = d_a1 * sigmoid_deriv(a1)
    d_W1 = X.T @ d_z1
    d_b1 = np.sum(d_z1, axis=0, keepdims=True)
    W2 -= lr * d_W2
    b2 -= lr * d_b2
    W1 -= lr * d_W1
    b1 -= lr * d_b1
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/50 | Loss: {loss:.4f}")

predictions = (a2 > 0.5).astype(int)
accuracy = np.mean(predictions == Y) * 100
print(f"Accuracy: {accuracy:.1f}%")

test = np.array([[5, 65]]) / np.array([10, 100])
z1t = sigmoid(test @ W1 + b1)
a2t = sigmoid(z1t @ W2 + b2)
print(f"Prediction for (5hrs, 65%): {'Pass' if a2t[0][0]>0.5 else 'Fail'} ({a2t[0][0]:.4f})")
```

Q3 — Transformer Components From Scratch

Algorithm

Step 1: Define input vectors X = [[1,0,1],[0,1,1],[1,1,0]].
Step 2: Implement Positional Encoding using sin/cos functions.
Step 3: Implement Scaled Dot-Product Attention: scores = $Q \cdot K^T/\sqrt{dk}$; apply softmax; multiply by V.
Step 4: Implement Multi-Head Attention (2 heads): split d_model, compute attention per head, concatenate.
Step 5: Implement Feed-Forward Network: ReLU(X·W1)·W2.
Step 6: Implement Layer Normalization.
Step 7: Combine into Transformer Encoder block; predict next vector using output linear layer.

Code

```
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / e_x.sum(axis=-1, keepdims=True)

def positional_encoding(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
            if i + 1 < d_model:
                PE[pos, i+1] = np.cos(pos / (10000 ** (i / d_model)))
    return PE

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    weights = softmax(scores)
    return weights @ V, weights

def multi_head_attention(X, num_heads=2):
    d_model = X.shape[-1]
    d_head = d_model // num_heads
    outputs = []
    for _ in range(num_heads):
        Wq = np.random.randn(d_model, d_head) * 0.1
        Wk = np.random.randn(d_model, d_head) * 0.1
        Wv = np.random.randn(d_model, d_head) * 0.1
        out, _ = scaled_dot_product_attention(X @ Wq, X @ Wk, X @ Wv)
        outputs.append(out)
    return np.concatenate(outputs, axis=-1)

def feed_forward(X, d_ff=16):
    d_model = X.shape[-1]
    W1 = np.random.randn(d_model, d_ff) * 0.1
    W2 = np.random.randn(d_ff, d_model) * 0.1
    return np.maximum(0, X @ W1) @ W2

def layer_norm(X, eps=1e-6):
    mean = X.mean(axis=-1, keepdims=True)
    std = X.std(axis=-1, keepdims=True)
    return (X - mean) / (std + eps)

def transformer_encoder(X):
    X = X + positional_encoding(X.shape[0], X.shape[1])
    attn_out = multi_head_attention(X)
    X = layer_norm(X + attn_out)
    ff_out = feed_forward(X)
    X = layer_norm(X + ff_out)
```

```
    return X

np.random.seed(42)
X = np.array([[1, 0, 1], [0, 1, 1], [1, 1, 0]], dtype=float)
output = transformer_encoder(X)
print("Output shape:", output.shape)
print("Output:\n", output)

Wo = np.random.randn(X.shape[1], X.shape[1]) * 0.1
next_vec = output[-1] @ Wo
print("Predicted next vector:", next_vec)
```

Q4 — MLP for XOR Problem

Algorithm

Step 1: Define XOR dataset: inputs [0,0],[0,1],[1,0],[1,1]; outputs [0,1,1,0].
Step 2: Initialize W1 (2→4), b1, W2 (4→1), b2.
Step 3: For each learning rate (0.01, 0.1), train for 10 epochs:
  Forward: a1 = sigmoid(X·W1 + b1); a2 = sigmoid(a1·W2 + b2).
  Backward: compute gradients; update weights.
Step 4: Observe convergence behavior at different learning rates.

Code

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
Y = np.array([[0],[1],[1],[0]], dtype=float)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_deriv(x):
    return x * (1 - x)

for lr in [0.01, 0.1]:
    np.random.seed(0)
    W1 = np.random.randn(2, 4)
    b1 = np.zeros((1, 4))
    W2 = np.random.randn(4, 1)
    b2 = np.zeros((1, 1))

    for epoch in range(10):
        z1 = X @ W1 + b1
        a1 = sigmoid(z1)
        z2 = a1 @ W2 + b2
        a2 = sigmoid(z2)
        loss = np.mean((Y - a2)**2)
        d2 = -(Y - a2) * sigmoid_deriv(a2)
        dW2 = a1.T @ d2
        db2 = d2.sum(axis=0)
        d1 = (d2 @ W2.T) * sigmoid_deriv(a1)
        dW1 = X.T @ d1
        db1 = d1.sum(axis=0)
        W2 -= lr * dW2
        b2 -= lr * db2
        W1 -= lr * dW1
        b1 -= lr * db1

    print(f"LR={lr} | Final Loss: {loss:.4f} | Predictions: {a2.round(3).flatten()}")
```

Q5 & Q6 — Transformer on Pixel Sequences

Algorithm

Step 1: Define pixel sequence [1,0,1,1,0]; set d_model=4.
Step 2: Tile pixel values into a 2D matrix and add positional encoding.
Step 3: Initialize Wq, Wk, Wv, Wo randomly.
Step 4: For each learning rate (0.001, 0.01), train for 10 epochs:
  Compute Q=X·Wq, K=X·Wk, V=X·Wv; compute scaled dot-product attention.
  Predict next pixel via output linear layer; compute MSE loss; update Wo.
Step 5: Report final loss and predicted next pixel value.

Code

```
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()

def positional_encoding(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
            if i+1 < d_model:
                PE[pos, i+1] = np.cos(pos / (10000 ** (i / d_model)))
    return PE

def scaled_dot_product_attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    weights = softmax(scores)
    return weights @ V

pixels = np.array([1, 0, 1, 1, 0], dtype=float)
d_model = 4
seq_len = len(pixels)

X = np.tile(pixels.reshape(-1, 1), (1, d_model))
X = X + positional_encoding(seq_len, d_model)

for lr in [0.001, 0.01]:
    np.random.seed(0)
    Wq = np.random.randn(d_model, d_model) * 0.1
    Wk = np.random.randn(d_model, d_model) * 0.1
    Wv = np.random.randn(d_model, d_model) * 0.1
    Wo = np.random.randn(d_model, 1) * 0.1

    for epoch in range(10):
        Q = X @ Wq
        K = X @ Wk
        V = X @ Wv
        attn_out = scaled_dot_product_attention(Q, K, V)
        pred = attn_out @ Wo
        target = pixels[-1]
        loss = np.mean((pred - target)**2)
        d_out = 2 * (pred - target) / len(pred)
        dWo = attn_out.T @ d_out
        Wo -= lr * dWo

    print(f"LR={lr} | Final Loss: {loss:.4f} | Predicted next pixel: {pred[-1][0]:.4f}")
```

Q7 — GAN Using Pixel Vectors

Algorithm

Step 1: Define real patterns: [[1,0,1,0],[0,1,0,1]].
Step 2: Build Generator: noise(2) → Dense(4, sigmoid) → fake pattern.
Step 3: Build Discriminator: input(4) → Dense(4, sigmoid) → Dense(1, sigmoid) → real/fake score.
Step 4: For each learning rate (0.001, 0.01), train for 10 epochs:
  Discriminator training: D_loss = -log(D(real)) - log(1 - D(fake)); backprop and update D_W1, D_W2.
  Generator training: G_loss = -log(D(G(z))); backprop discriminator gradients all the way to G; update G_W1.
Step 5: Generate new patterns from random noise and compare outputs across learning rates.

Code

```
import numpy as np

real_data = np.array([[1,0,1,0],[0,1,0,1]], dtype=float)

def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -10, 10)))

def sigmoid_deriv(x):
    return x * (1 - x)

for lr in [0.001, 0.01]:
    np.random.seed(42)
    G_W1 = np.random.randn(2, 4) * 0.1
    G_b1 = np.zeros((1, 4))
    D_W1 = np.random.randn(4, 4) * 0.1
    D_b1 = np.zeros((1, 4))
    D_W2 = np.random.randn(4, 1) * 0.1
    D_b2 = np.zeros((1, 1))

    for epoch in range(10):
        noise = np.random.randn(2, 2)
        fake = sigmoid(noise @ G_W1 + G_b1)

        real_h = sigmoid(real_data @ D_W1 + D_b1)
        real_out = sigmoid(real_h @ D_W2 + D_b2)
        fake_h = sigmoid(fake @ D_W1 + D_b1)
        fake_out = sigmoid(fake_h @ D_W2 + D_b2)

        d_loss = -np.mean(np.log(real_out + 1e-8)) - np.mean(np.log(1 - fake_out + 1e-8))

        d_real = -(1 - real_out) * sigmoid_deriv(real_out)
        d_fake = fake_out * sigmoid_deriv(fake_out)
        dD_W2 = real_h.T @ d_real / 2 + fake_h.T @ d_fake / 2
        dD_b2 = (d_real + d_fake).mean(axis=0)
        d_real_h = d_real @ D_W2.T * sigmoid_deriv(real_h)
        d_fake_h = d_fake @ D_W2.T * sigmoid_deriv(fake_h)
        dD_W1 = real_data.T @ d_real_h / 2 + fake.T @ d_fake_h / 2
        dD_b1 = (d_real_h + d_fake_h).mean(axis=0)
        D_W2 -= lr * dD_W2
        D_b2 -= lr * dD_b2
        D_W1 -= lr * dD_W1
        D_b1 -= lr * dD_b1

        fake_h2 = sigmoid(fake @ D_W1 + D_b1)
        fake_out2 = sigmoid(fake_h2 @ D_W2 + D_b2)
        g_loss = -np.mean(np.log(fake_out2 + 1e-8))

        d_g_out = -(1 / (fake_out2 + 1e-8)) * sigmoid_deriv(fake_out2)
        d_g_h = d_g_out @ D_W2.T * sigmoid_deriv(fake_h2)
        d_g_fake = d_g_h @ D_W1.T * sigmoid_deriv(fake)
```

```
        G_W1 -= lr * noise.T @ d_g_fake
        G_b1 -= lr * d_g_fake.mean(axis=0, keepdims=True)

    print(f"LR={lr} | D_loss: {d_loss:.4f} | G_loss: {g_loss:.4f}")
    noise = np.random.randn(2, 2)
    generated = sigmoid(noise @ G_W1 + G_b1)
    print(f"Generated patterns:\n{generated.round(3)}")
```

Q8 — Self-Attention on Pixel Vectors

Algorithm

Step 1: Define pixel vector [1,0,1,0,1] (d_model=5).
Step 2: Initialize Wq, Wk, Wv randomly.
Step 3: For each learning rate (0.001, 0.01), run 10 steps:
  Compute Q = pixels·Wq, K = pixels·Wk, V = pixels·Wv.
  Compute attention scores = $Q \cdot K^T / \sqrt{d_model}$; apply softmax.
  Compute attention output = weights · V.
  Compute loss; update Wv via gradient descent.
Step 4: Identify most important pixel (highest attention weight).

Code

```
import numpy as np

def softmax(x):
    e_x = np.exp(x - np.max(x))
    return e_x / e_x.sum()

pixels = np.array([[1, 0, 1, 0, 1]], dtype=float)
d_model = 5

np.random.seed(42)
Wq = np.random.randn(d_model, d_model) * 0.1
Wk = np.random.randn(d_model, d_model) * 0.1
Wv = np.random.randn(d_model, d_model) * 0.1

for lr in [0.001, 0.01]:
    Wv = np.random.randn(d_model, d_model) * 0.1
    for step in range(10):
        Q = pixels @ Wq
        K = pixels @ Wk
        V = pixels @ Wv
        scores = Q @ K.T / np.sqrt(d_model)
        weights = softmax(scores[0])
        attn_out = weights @ V
        loss = np.mean(attn_out**2)
        d_attn = 2 * attn_out / attn_out.size
        Wv -= lr * pixels.T @ (weights.reshape(1,-1) * d_attn.reshape(1,-1))

    print(f"LR={lr} | Attention weights: {weights.round(4)}")
    most_important = np.argmax(weights)
    print(f"Most important pixel index: {most_important} (value: {pixels[0,
most_important]})")
```

Q9 — RNN on Pixel Sequences

Algorithm

Step 1: Define 3 pixel rows as sequence: Row1=[1,1,1], Row2=[0,1,0], Row3=[1,1,1].
Step 2: Initialize Wxh (3→4), Whh (4→4), bh, Why (4→1), by randomly.
Step 3: For each learning rate (0.01, 0.1), train for 10 epochs:
  Forward: iterate over rows; h = tanh(row·Wxh + h·Whh + bh); y_pred = sigmoid(h·Why + by).
  Compute MSE loss; backpropagate; update Why and by.
Step 4: Report loss and prediction.

Code

```
import numpy as np

rows = np.array([[1,1,1],[0,1,0],[1,1,1]], dtype=float)
label = np.array([1.0])
input_size = 3
hidden_size = 4
output_size = 1

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

for lr in [0.01, 0.1]:
    np.random.seed(42)
    Wxh = np.random.randn(input_size, hidden_size) * 0.1
    Whh = np.random.randn(hidden_size, hidden_size) * 0.1
    bh = np.zeros((1, hidden_size))
    Why = np.random.randn(hidden_size, output_size) * 0.1
    by = np.zeros((1, output_size))

    for epoch in range(10):
        h = np.zeros((1, hidden_size))
        for t in range(len(rows)):
            h = np.tanh(rows[t].reshape(1,-1) @ Wxh + h @ Whh + bh)
        y_pred = sigmoid(h @ Why + by)
        loss = np.mean((label - y_pred)**2)
        dy = -(label - y_pred) * y_pred * (1 - y_pred)
        Why -= lr * h.T @ dy
        by -= lr * dy

    print(f"LR={lr} | Final Loss: {loss:.4f} | Prediction: {y_pred[0][0]:.4f}")
```

Q10 — CNN for Cross Pattern Recognition

Algorithm

Step 1: Define 4×4 cross pattern flattened; reshape to (1,1,4,4).
Step 2: Implement manual conv2d and 2×2 max pooling.
Step 3: Build mini-CNN: conv → relu → maxpool → flatten → Dense(1, sigmoid).
Step 4: For each LR (0.01, 0.1), train for 10 epochs; update W_fc via gradient.
Step 5: Print loss, prediction, and display activation maps.

Code

```
import numpy as np

pattern = np.array([0,1,0,0,1,1,1,0,0,1,0,0,0,0,0,0], dtype=float).reshape(1,1,4,4)
label = np.array([1.0])
b_conv = np.zeros(1)
b_fc = np.zeros(1)

def relu(x): return np.maximum(0, x)
def sigmoid(x): return 1 / (1 + np.exp(-x))

def conv2d(x, k):
    h_out = x.shape[2] - k.shape[2] + 1
    w_out = x.shape[3] - k.shape[3] + 1
    out = np.zeros((x.shape[0], k.shape[0], h_out, w_out))
    for i in range(h_out):
        for j in range(w_out):
            out[0, 0, i, j] = np.sum(x[0, 0, i:i+2, j:j+2] * k[0, 0])
    return out

def maxpool(x):
    h_out = x.shape[2] // 2
    w_out = x.shape[3] // 2
    out = np.zeros((x.shape[0], x.shape[1], h_out, w_out))
    for i in range(h_out):
        for j in range(w_out):
            out[0, 0, i, j] = np.max(x[0, 0, i*2:(i+1)*2, j*2:(j+1)*2])
    return out

for lr in [0.01, 0.1]:
    np.random.seed(42)
    kernel = np.random.randn(1,1,2,2) * 0.1
    W_fc = np.random.randn(9, 1) * 0.1

    for epoch in range(10):
        conv_out = relu(conv2d(pattern, kernel) + b_conv)
        pool_out = maxpool(conv_out)
        flat = pool_out.flatten()
        y_pred = sigmoid(flat @ W_fc + b_fc)
        loss = np.mean((label - y_pred)**2)
        d_pred = -(label - y_pred) * y_pred * (1 - y_pred)
        W_fc -= lr * flat.reshape(-1,1) * d_pred

    print(f"LR={lr} | Loss: {loss:.4f} | Prediction: {y_pred[0]:.4f}")
    print(f"Activation map (conv):\n{conv_out[0,0].round(3)}")
```

Q11 & Q12 — Noise Removal Using Autoencoder

Algorithm

Step 1: Define clean image [1,1,1,1,0,1,1,1,1] and noisy [1,0,1,1,1,1,1,0,1].
Step 2: Initialize encoder W_enc (9→4) and decoder W_dec (4→9).
Step 3: For each LR (0.001, 0.01), train for 10 epochs:
  Forward: h = sigmoid(noisy·W_enc + b_enc); output = sigmoid(h·W_dec + b_dec).
  Loss = MSE(clean, output).
  Backward: compute gradients and update W_enc, W_dec, biases.
Step 4: Report loss start vs end; print reconstructed output.

Code

```
import numpy as np

clean = np.array([[1,1,1,1,0,1,1,1,1]], dtype=float)
noisy = np.array([[1,0,1,1,1,1,1,0,1]], dtype=float)
input_dim = 9
hidden_dim = 4

def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x): return x * (1 - x)

for lr in [0.001, 0.01]:
    np.random.seed(42)
    W_enc = np.random.randn(input_dim, hidden_dim) * 0.1
    b_enc = np.zeros(hidden_dim)
    W_dec = np.random.randn(hidden_dim, input_dim) * 0.1
    b_dec = np.zeros(input_dim)
    losses = []

    for epoch in range(10):
        h = sigmoid(noisy @ W_enc + b_enc)
        output = sigmoid(h @ W_dec + b_dec)
        loss = np.mean((clean - output)**2)
        losses.append(loss)
        d_out = -(clean - output) * sigmoid_deriv(output)
        dW_dec = h.T @ d_out
        db_dec = d_out.flatten()
        d_h = d_out @ W_dec.T * sigmoid_deriv(h)
        dW_enc = noisy.T @ d_h
        db_enc = d_h.flatten()
        W_dec -= lr * dW_dec
        b_dec -= lr * db_dec
        W_enc -= lr * dW_enc
        b_enc -= lr * db_enc

    print(f"LR={lr} | Loss Start: {losses[0]:.4f} | Loss End: {losses[-1]:.4f}")
    print(f"Reconstructed: {output.round(3)}")
```

Q13 — MLP for Pixel-Based Image Classification (Bright/Dark)

Algorithm

Step 1: Define bright [255,255,255,255]→1 and dark [10,10,10,10]→0.
Step 2: Normalize pixels by dividing by 255.
Step 3: Initialize W1 (4→4), b1, W2 (4→1), b2.
Step 4: For each LR (0.001, 0.01), train for 10 epochs using sigmoid + MSE + backprop.
Step 5: Predict class for [128,128,128,128]; analyze decision boundary.

Code

```
import numpy as np

X = np.array([[255,255,255,255],[10,10,10,10]], dtype=float) / 255.0
Y = np.array([[1],[0]], dtype=float)

def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x): return x * (1 - x)

for lr in [0.001, 0.01]:
    np.random.seed(42)
    W1 = np.random.randn(4, 4) * 0.1
    b1 = np.zeros(4)
    W2 = np.random.randn(4, 1) * 0.1
    b2 = np.zeros(1)

    for epoch in range(10):
        a1 = sigmoid(X @ W1 + b1)
        a2 = sigmoid(a1 @ W2 + b2)
        loss = np.mean((Y - a2)**2)
        d2 = -(Y - a2) * sigmoid_deriv(a2)
        dW2 = a1.T @ d2
        d1 = (d2 @ W2.T) * sigmoid_deriv(a1)
        dW1 = X.T @ d1
        W2 -= lr * dW2
        W1 -= lr * dW1

    preds = (a2 > 0.5).astype(int)
    print(f"LR={lr} | Loss: {loss:.4f} | Predictions: {preds.flatten()} | Expected: [1 0]")

test = np.array([[128,128,128,128]]) / 255.0
a1t = sigmoid(test @ W1 + b1)
a2t = sigmoid(a1t @ W2 + b2)
print(f"New [128,128,128,128]: {'Bright' if a2t[0][0]>0.5 else 'Dark'} ({a2t[0][0]:.4f})")
```

Q14 — Edge Detection Using CNN

Algorithm

Step 1: Define 3×3 grayscale image [[10,10,10],[0,0,0],[10,10,10]].
Step 2: Apply Sobel kernel manually via convolution to detect horizontal edges.
Step 3: Build a learnable CNN layer; initialize kernel randomly.
Step 4: For each LR (0.01, 0.5), train for 10 epochs:
  Forward: convolve image with kernel; compute MSE loss.
  Backward: compute gradient w.r.t. kernel; update kernel.
Step 5: Compare feature maps at both learning rates.

Code

```
import numpy as np

image = np.array([10,10,10,0,0,0,10,10,10], dtype=float).reshape(3,3)
sobel_kernel = np.array([[-1,0,1],[-2,0,2],[-1,0,1]])
conv_result = np.sum(image * sobel_kernel)
print(f"Manual Sobel convolution result: {conv_result}")

def conv2d_full(img, k):
    h, w = img.shape
    kh, kw = k.shape
    out_h = h - kh + 1
    w_out = w - kw + 1
    out = np.zeros((out_h, w_out))
    for i in range(out_h):
        for j in range(w_out):
            out[i,j] = np.sum(img[i:i+kh, j:j+kw] * k)
    return out

target_output = np.array([[0.0]])

for lr in [0.01, 0.5]:
    np.random.seed(42)
    kernel_learnable = np.random.randn(3,3) * 0.1
    for epoch in range(10):
        feature_map = conv2d_full(image / 10.0, kernel_learnable)
        loss = np.mean((feature_map - target_output)**2)
        d_kernel = np.zeros_like(kernel_learnable)
        for i in range(feature_map.shape[0]):
            for j in range(feature_map.shape[1]):
                d_kernel += 2*(feature_map[i,j]-target_output[0,0])*(image[i:i+3,
j:j+3]/10.0)
        kernel_learnable -= lr * d_kernel

    print(f"LR={lr} | Final Loss: {loss:.4f}")
    print(f"Feature map:\n{feature_map.round(3)}")
```

Q15 & Q17 — CNN on Digit-like Pixel Patterns (0 and 1)

Algorithm

Step 1: Define 5×5 Digit-0 and Digit-1 pixel patterns; reshape to (1,1,5,5).
Step 2: Implement conv2d (3×3 kernel) and 2×2 maxpool manually.
Step 3: Build: Conv(relu) → MaxPool → Flatten → Dense(1, sigmoid).
Step 4: For each LR (0.01, 0.1), train for 10 epochs; update W_fc via gradient.
Step 5: Classify new pattern; explain feature extraction.

Code

```
import numpy as np

digit_0 = np.array([1,1,1,1,1,1,0,0,0,1,1,0,0,0,1,1,0,0,0,1,1,1,1,1,1],
                   dtype=float).reshape(1,1,5,5)
digit_1 = np.array([0,0,1,0,0,0,1,1,0,0,1,0,1,0,0,0,0,1,0,0,1,1,1,1,1],
                   dtype=float).reshape(1,1,5,5)
X = np.concatenate([digit_0, digit_1], axis=0)
Y = np.array([0, 1], dtype=float)

def relu(x): return np.maximum(0, x)
def sigmoid(x): return 1 / (1 + np.exp(-x))

def conv2d(img, kernel):
    kh, kw = kernel.shape[2], kernel.shape[3]
    h_out = img.shape[2] - kh + 1
    w_out = img.shape[3] - kw + 1
    out = np.zeros((img.shape[0], kernel.shape[0], h_out, w_out))
    for n in range(img.shape[0]):
        for k in range(kernel.shape[0]):
            for i in range(h_out):
                for j in range(w_out):
                    out[n,k,i,j] = np.sum(img[n,0,i:i+kh,j:j+kw]*kernel[k,0])
    return out

def maxpool2d(x, size=2):
    h_out = x.shape[2] // size
    w_out = x.shape[3] // size
    out = np.zeros((x.shape[0], x.shape[1], h_out, w_out))
    for n in range(x.shape[0]):
        for c in range(x.shape[1]):
            for i in range(h_out):
                for j in range(w_out):
                    out[n,c,i,j] = np.max(x[n,c,i*size:(i+1)*size,j*size:(j+1)*size])
    return out

np.random.seed(42)
kernel = np.random.randn(1,1,3,3) * 0.1
conv_out = relu(conv2d(X, kernel))
pool_out = maxpool2d(conv_out)
flat_size = pool_out.reshape(2,-1).shape[1]

for lr in [0.01, 0.1]:
    np.random.seed(42)
    kernel = np.random.randn(1,1,3,3) * 0.1
    W_fc = np.random.randn(flat_size, 1) * 0.1
    b_fc = np.zeros(1)
    for epoch in range(10):
        conv_out = relu(conv2d(X, kernel))
        pool_out = maxpool2d(conv_out)
        flat = pool_out.reshape(2,-1)
        y_pred = sigmoid(flat @ W_fc + b_fc).flatten()
        loss = np.mean((Y - y_pred)**2)
        d_out = -(Y - y_pred) * y_pred * (1 - y_pred)
        W_fc -= lr * flat.T @ d_out.reshape(-1,1)
```

```
    print(f"LR={lr} | Loss: {loss:.4f} | Pred: {y_pred.round(3)}")

new_pattern = digit_0[0:1]
conv_out = relu(conv2d(new_pattern, kernel))
pool_out = maxpool2d(conv_out)
flat = pool_out.reshape(1,-1)
pred = sigmoid(flat @ W_fc + b_fc)
print(f"New pattern: {'Digit 1' if pred[0][0]>0.5 else 'Digit 0'} ({pred[0][0]:.4f})")
```

Q16 — Variational Autoencoder (VAE) on Pixel Patterns

Algorithm

Step 1: Define 4 digit-like 4×4 patterns as 16-dimensional vectors.
Step 2: Encoder: input(16) → Dense(8, sigmoid) → two output heads: mu(4) and log_var(4).
Step 3: Reparameterization trick: z = mu + epsilon * exp(0.5 * log_var) [epsilon ~ N(0,1)].
Step 4: Decoder: z(4) → Dense(8, sigmoid) → output(16, sigmoid) — reconstructed pattern.
Step 5: Total Loss = Reconstruction Loss + KL Divergence:
  Recon Loss = MSE(original, reconstructed)
  KL Loss = -0.5 * mean(1 + log(sigma^2) - mu^2 - sigma^2)
        = -0.5 * mean(1 + log_var - mu^2 - exp(log_var))
Step 6: Backprop through decoder; update W_dec2 and W_dec1.
Step 7: For each LR (0.001, 0.01), train for 10 epochs.
Step 8: Sample z ~ N(0,1) and decode to generate new digit-like patterns.

Code

```
import numpy as np

patterns = np.array([
    [1,1,1,1,1,0,0,1,1,0,0,1,1,1,1,1],
    [0,1,0,0,1,1,0,0,0,1,0,0,1,1,1,0],
    [1,1,1,0,0,0,1,0,1,1,1,0,0,0,1,0],
    [1,1,1,1,0,0,1,0,0,1,0,0,1,1,1,1]
], dtype=float)

input_dim = 16
latent_dim = 4

def sigmoid(x): return 1 / (1 + np.exp(-np.clip(x, -10, 10)))
def sigmoid_deriv(x): return x * (1 - x)

for lr in [0.001, 0.01]:
    np.random.seed(42)
    W_enc  = np.random.randn(input_dim, 8) * 0.1
    W_mu   = np.random.randn(8, latent_dim) * 0.1
    W_logv = np.random.randn(8, latent_dim) * 0.1
    W_dec1 = np.random.randn(latent_dim, 8) * 0.1
    W_dec2 = np.random.randn(8, input_dim) * 0.1

    for epoch in range(10):
        h_enc  = sigmoid(patterns @ W_enc)
        mu     = h_enc @ W_mu
        log_var = h_enc @ W_logv

        eps = np.random.randn(*mu.shape)
        z   = mu + eps * np.exp(0.5 * log_var)

        h_dec = sigmoid(z @ W_dec1)
        recon = sigmoid(h_dec @ W_dec2)

        recon_loss = np.mean((patterns - recon) ** 2)

        kl_loss = -0.5 * np.mean(1 + log_var - mu**2 - np.exp(log_var))

        total_loss = recon_loss + 0.001 * kl_loss

        d_recon  = -(patterns - recon) * sigmoid_deriv(recon)
        dW_dec2  = h_dec.T @ d_recon
        W_dec2  -= lr * dW_dec2

        d_h_dec  = d_recon @ W_dec2.T * sigmoid_deriv(h_dec)
```

```
        dW_dec1  = z.T @ d_h_dec
        W_dec1  -= lr * dW_dec1

        d_z      = d_h_dec @ W_dec1.T
        d_mu     = d_z + 0.001 * mu
        d_logv   = d_z * eps * 0.5 * np.exp(0.5 * log_var) + 0.001 * 0.5 * (np.exp(log_var) -
1)
        W_mu    -= lr * h_enc.T @ d_mu
        W_logv  -= lr * h_enc.T @ d_logv

        if (epoch + 1) % 5 == 0:
            print(f"  LR={lr} | Epoch {epoch+1} | Recon: {recon_loss:.4f} | KL: {kl_loss:.4f}
| Total: {total_loss:.4f}")

    z_sample  = np.random.randn(2, latent_dim)
    h_gen     = sigmoid(z_sample @ W_dec1)
    generated = sigmoid(h_gen @ W_dec2)
    print(f"Generated samples (LR={lr}):\n{generated.round(3)}")
```

Q18 — Transformer Encoder for Review Classification

Algorithm

Step 1: Define 4 reviews as 3D tensor (4 samples × 3 words × 4-dim vectors); labels [1,1,0,0].

Step 2: Apply Positional Encoding to input.

Step 3: Transformer Encoder Block: Multi-Head Self-Attention → Add & LayerNorm → Feed Forward → Add & LayerNorm.

Step 4: Global Average Pooling over sequence dimension.

Step 5: Linear classifier → sigmoid → binary cross-entropy loss.

Step 6: For each LR (0.001, 0.01), train for 10 epochs; report accuracy.

Step 7: Predict class for a new review vector.

Code

```
import numpy as np

X = np.array([
    [[1,0,1,0],[1,1,1,0],[1,0,1,1]],
    [[0,1,1,1],[1,1,0,1],[1,1,1,1]],
    [[0,0,0,1],[0,1,0,0],[0,0,1,0]],
    [[1,0,0,0],[0,0,1,0],[0,1,0,0]]
], dtype=float)
Y = np.array([1, 1, 0, 0], dtype=float)

def softmax(x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / e_x.sum(axis=-1, keepdims=True)

def positional_encoding(seq_len, d_model):
    PE = np.zeros((seq_len, d_model))
    for pos in range(seq_len):
        for i in range(0, d_model, 2):
            PE[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
            if i+1 < d_model:
                PE[pos, i+1] = np.cos(pos / (10000 ** (i / d_model)))
    return PE

def layer_norm(x, eps=1e-6):
    return (x - x.mean(axis=-1, keepdims=True)) / (x.std(axis=-1, keepdims=True) + eps)

def attention(Q, K, V):
    d_k = Q.shape[-1]
    scores = Q @ K.transpose(0, 2, 1) / np.sqrt(d_k)
    weights = softmax(scores)
    return weights @ V

n_samples, seq_len, d_model = X.shape
PE = positional_encoding(seq_len, d_model)
X_pe = X + PE

for lr in [0.001, 0.01]:
    np.random.seed(42)
    Wq = np.random.randn(d_model, d_model) * 0.1
    Wk = np.random.randn(d_model, d_model) * 0.1
    Wv = np.random.randn(d_model, d_model) * 0.1
    W_ff1 = np.random.randn(d_model, 8) * 0.1
    W_ff2 = np.random.randn(8, d_model) * 0.1
    W_cls = np.random.randn(d_model, 1) * 0.1
    b_cls = np.zeros(1)

    for epoch in range(10):
        Q = X_pe @ Wq
        K = X_pe @ Wk
        V = X_pe @ Wv
```

```
        attn_out = attention(Q, K, V)
        X_attn = layer_norm(X_pe + attn_out)
        ff_out = np.maximum(0, X_attn @ W_ff1) @ W_ff2
        X_enc = layer_norm(X_attn + ff_out)
        pooled = X_enc.mean(axis=1)
        logits = (1 / (1 + np.exp(-(pooled @ W_cls + b_cls)))).flatten()
        loss = -np.mean(Y*np.log(logits+1e-8) + (1-Y)*np.log(1-logits+1e-8))
        d_logits = (logits - Y) / n_samples
        W_cls -= lr * pooled.T @ d_logits.reshape(-1,1)

    preds = (logits > 0.5).astype(int)
    acc = np.mean(preds == Y) * 100
    print(f"LR={lr} | Loss: {loss:.4f} | Accuracy: {acc:.0f}%")

new_review = np.array([[[0.5,0.5,0.5,0.5],[0.5,0.5,0.5,0.5],[0.5,0.5,0.5,0.5]]])
new_pe = new_review + PE
Q = new_pe @ Wq
K = new_pe @ Wk
V = new_pe @ Wv
attn = attention(Q, K, V)
enc = layer_norm(new_pe + attn)
pooled = enc.mean(axis=1)
pred = (1 / (1 + np.exp(-(pooled @ W_cls + b_cls))))
print(f"New review: {'Positive' if pred[0][0]>0.5 else 'Negative'} ({pred[0][0]:.4f})")
```